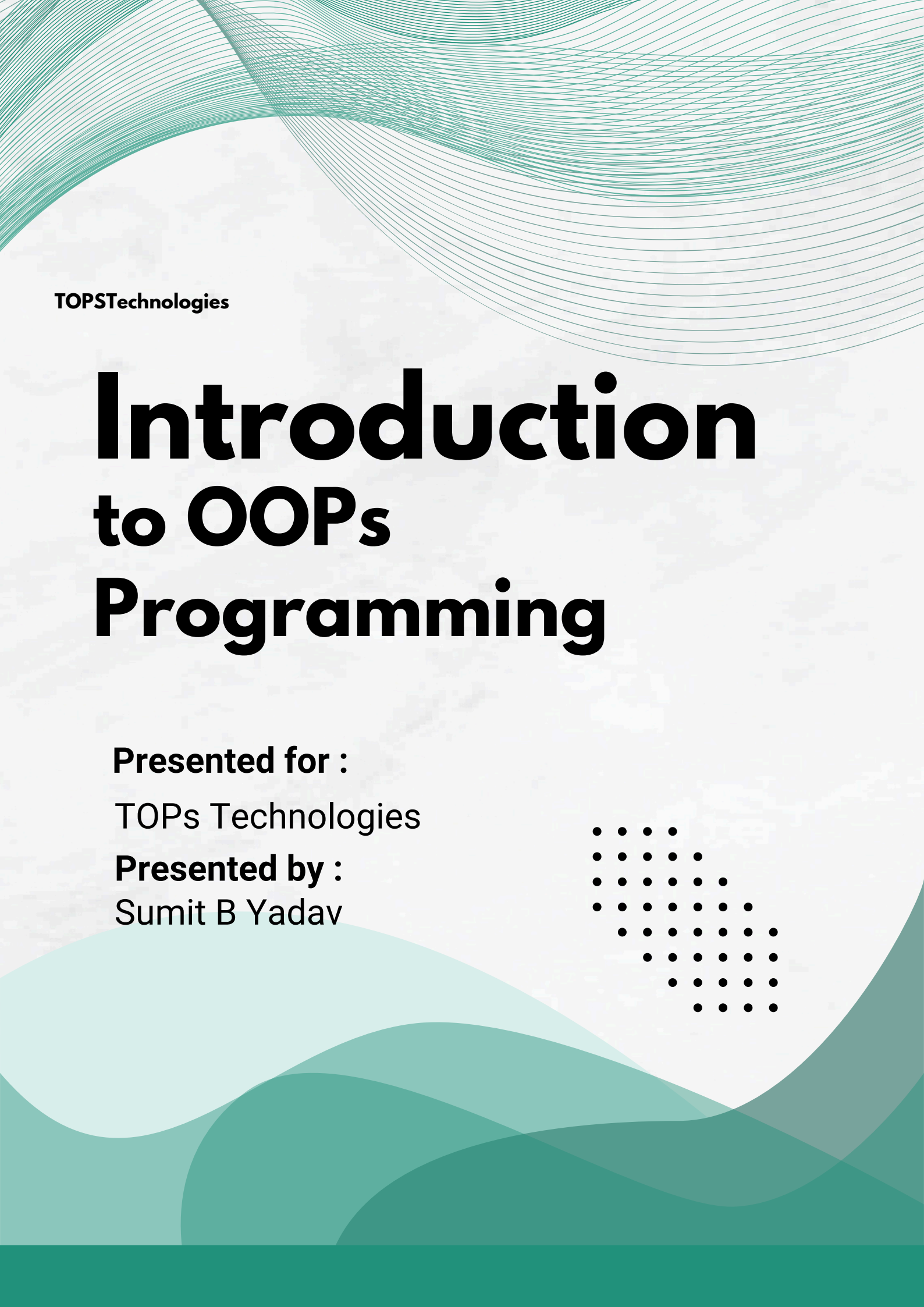




TOPSTechnologies

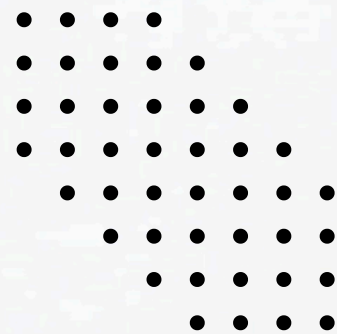
Introduction to OOPs Programming

Presented for :

TOPs Technologies

Presented by :

Sumit B Yadav



Basic Structure of a C++ Program

Que 1

An array is a collection of elements of the same type, stored in contiguous memory locations. Arrays allow you to store multiple values in a single variable, and these values can be accessed using an index. The main advantage of using arrays is that they provide an efficient way to handle collections of data when the number of elements is known in advance.

1. Single-Dimensional Arrays

A single-dimensional array (also called a 1D array) is a linear collection of elements, all of the same data type, arranged in a single row. You can think of it as a list of elements where each element can be accessed via its index.

2. Multi-Dimensional Arrays

A multi-dimensional array is an array of arrays. It can have more than one dimension, and the elements are stored in a table-like structure. The most common multi-dimensional array is a two-dimensional array (2D array), which is like a matrix (a table with rows and columns). You can also have three-dimensional arrays (3D arrays), and so on.

Two-Dimensional Array (2D Array):

A 2D array can be thought of as a matrix with rows and columns.

Basic Structure of a C++ Program

Que 2

String Handling in C++

In C++, strings are used to store sequences of characters. String handling refers to the various operations you can perform on strings, such as initializing, manipulating, and processing them. C++ provides two main ways to handle strings:

1. C-Style Strings (using character arrays)
2. C++ Standard Library Strings (using the `std::string` class)

1. C-Style Strings in C++

A C-style string is an array of characters terminated by a null character (`'\0'`). C-style strings are low-level representations of strings in C++, and they are similar to how strings are represented in the C programming language.

2. C++ Standard Library Strings (`std::string`)

The C++ Standard Library provides a higher-level string class called `std::string` in the `<string>` header. This class abstracts away the complexities of C-style strings and provides a rich set of built-in functions for string manipulation.

Basic Structure of a C++ Program

Example of C- style in C++

Do-while Loop

```
#include <iostream>  
using namespace std;
```

```
int main() {
```

```
// Declare and initialize a C-style string
```

```
char str[] = "Hello, World!"; // No need to specify the size, it is inferred
```

```
// Access and print the string
```

```
cout << str << endl; // Output: Hello, World!
```

```
// Get the length of the C-style string
```

```
int length = 0;
```

```
while (str[length] != '\0') {
```

```
length++; // Counting the characters until the null terminator  
}
```

```
cout << "Length of the string: " << length << endl; // Output: Length of  
the string: 13
```

```
return 0;
```

```
}
```

Basic Structure of a C++ Program

Example of Library string

```
#include <iostream>
#include <string> // Required for std::string

using namespace std;

int main() {
    // Declare a string
    string greeting = "Hello, World!";

    // Print the string
    cout << greeting << endl; // Output: Hello, World!

    // Get the length of the string
    cout << "Length of the string: " << greeting.length() << endl; // Output:
    13

    // Change the first character
    greeting[0] = 'h'; // Change 'H' to 'h'

    // Print the modified string
    cout << "Modified string: " << greeting << endl; // Output: hello, World!

    return 0;
}
```

Basic Structure of a C++ Program

Que 3

1. Initializing a Single-Dimensional (1D) Array

A 1D array is a simple list of elements, all of the same type, arranged in a single row.

Static Initialization of a 1D Array

Static initialization means that the array's values are set at the time of declaration.

Dynamic Initialization of a 1D Array

Dynamic initialization is when the values of the array are assigned during program execution.

2. Initializing a Two-Dimensional (2D) Array

A 2D array can be thought of as a table with rows and columns. The syntax for a 2D array is similar to that of a 1D array, but it includes two indices – one for the row and one for the column.

Basic Structure of a C++ Program

Static Initialization with Specified Size

```
#include <iostream>
using namespace std;

main() {
// Declare and initialize a 2D array with 2 rows and 3 columns
int arr[2][3] = { {1, 2, 3}, {4, 5, 6} };

// Print the 2D array elements
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}
// Output:
// 1 2 3
// 4 5 6
}
```

Basic Structure of a C++ Program

Dynamic Initialization of a 2D Array

```
#include <iostream>
using namespace std;

main() {
    // Declare a 2D array with 2 rows and 3 columns
    int arr[2][3];

    // Dynamically initialize the array elements
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            arr[i][j] = (i + 1) * (j + 1); // Dynamically assign values
        }
    }

    // Print the 2D array elements
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    // Output:
    // 1 2 3
    // 2 4 6

}
```

Basic Structure of a C++ Program

Que 4

String Operations and Functions in C++

In C++, string handling is primarily done using two approaches:

- 1. C-Style Strings: These are arrays of characters terminated by a null character ('\0'). They are essentially the same as in the C programming language.*
- 2. C++ std::string: This is part of the C++ Standard Library, and it's a higher-level class that provides a variety of functions to handle and manipulate strings.*

Here, we'll focus on std::string operations and functions because they are more commonly used in modern C++ programming. The std::string class provides an extensive set of member functions for string manipulation. It abstracts away the complexities of handling memory and provides a lot of convenience.

1. String Initialization

You can initialize a std::string in several ways:

2. String Length

To find the length (number of characters) of a string, use the length() or size() function:

3. String Concatenation

You can concatenate strings using the + operator or the append() function: